2026-05-06-p1-f17-smart-file-editing-design

Phase 1 / Feature 17 — Smart File Editing

Date: 2026-05-06 **Status:** Approved (auto-approved per programme cadence) **Programme:** CLI-Agent Fusion — Phase 1 port from claude-code

---

1. Goal

Ship real, end-to-end **search-replace block editing** for the HelixCode CLI agent. F17 adds a `smart_edit` tool (and a thin `/edit` slash for inspection) that consumes one or more **search-replace blocks** in a single string payload, applies them transactionally to one or more files via the existing F08 multiedit transaction layer, and returns both the updated file content and a unified diff so the caller can self-check that the change actually landed.

Three concrete user surfaces ship together:

1. `smart_edit` **tool** — registered into the F09/F13 tool registry. Single parameter `prompt` (string) carrying one or more blocks of the form

   ```
   path/to/file.ext
   <<<<<<< SEARCH
   <old text — must match the current file content literally>
   =======
   <new text — replaces the matched region>
   >>>>>>> REPLACE
   ```

   Multiple blocks per file and multiple files per `prompt` are both allowed. Standard claude-code/aider delimiter triplet (Q1=B). Returns a `SmartEditResult` with per-file `Applied` / `Diff` / `NewContent` / `Error` fields. **All-or-nothing across the whole prompt** (Q3=B; one failed block aborts the entire commit and writes nothing).

2. `/edit` **slash command** (Q5=A) — `/edit status` / `/edit diff` / `/edit dry-run <path>` / `/edit commit <path>`. The slash is for **inspecting the most recent smart-edit attempt** plus running a one-shot dry-run from a file containing blocks. **No cobra subcommand** — slash only.

3. **Verification dual-output** (Q2=C) — every apply re-reads the file from disk after the multiedit commit AND returns the unified diff between the pre-apply and post-apply content. The caller sees both `NewContent` (post-write re-read, not the in-memory plan) and `Diff` (computed from before/after). Maximum visibility for agent self-check; the LLM can grep its own `Diff` to confirm the change actually happened on disk.

The conflict-detection model is **lenient** (Q4=B): no mtime check, no hash check, no transaction-time snapshot. The applier re-searches each SEARCH block in the **current** file content at apply time. If the literal SEARCH text is absent (or appears more than once), the block fails with a clear error. This handles the common case where an external tool reformatted the file between the agent's plan and the apply (e.g., `gofmt` ran in a watcher) without aborting the whole edit just because the file changed.

The atomicity model is **transactional via multiedit** (Q3=B): F17 is **non-breaking** — it builds `SmartEditCommit` ON TOP of the existing `internal/tools/multiedit/` package (which is transactional from F08; see `MultiFileEditor.BeginEdit` / `Commit` / `Rollback` in `internal/tools/multiedit/multiedit.go`). Per-file failure during commit triggers `MultiFileEditor.Rollback`, which restores backups for any files already written. Single-file edits use a multiedit transaction with one `FileEdit`; multi-file edits use one transaction with N `FileEdit` entries.

Out of scope for v1: fuzzy SEARCH matching (typos in old text), regex SEARCH patterns, partial-line matching, binary-aware diffs, auto-fix-on-conflict, structural-AST edits, undo history (already covered by multiedit's `Rollback` for the in-flight transaction; multi-step undo across separate prompts is F17.5).

Anti-bluff: a `smart_edit` tool that reports `applied=true` without the file content actually changing on disk, OR reports a successful apply when SEARCH was never found, OR returns a `Diff` that doesn't match what's actually on disk after the write — each is a critical defect (§5.2). The single largest bluff vector for F17 is "tool says edit applied but file unchanged" — looks correct in compilation, fails on any disk re-read.

---

2. Architecture

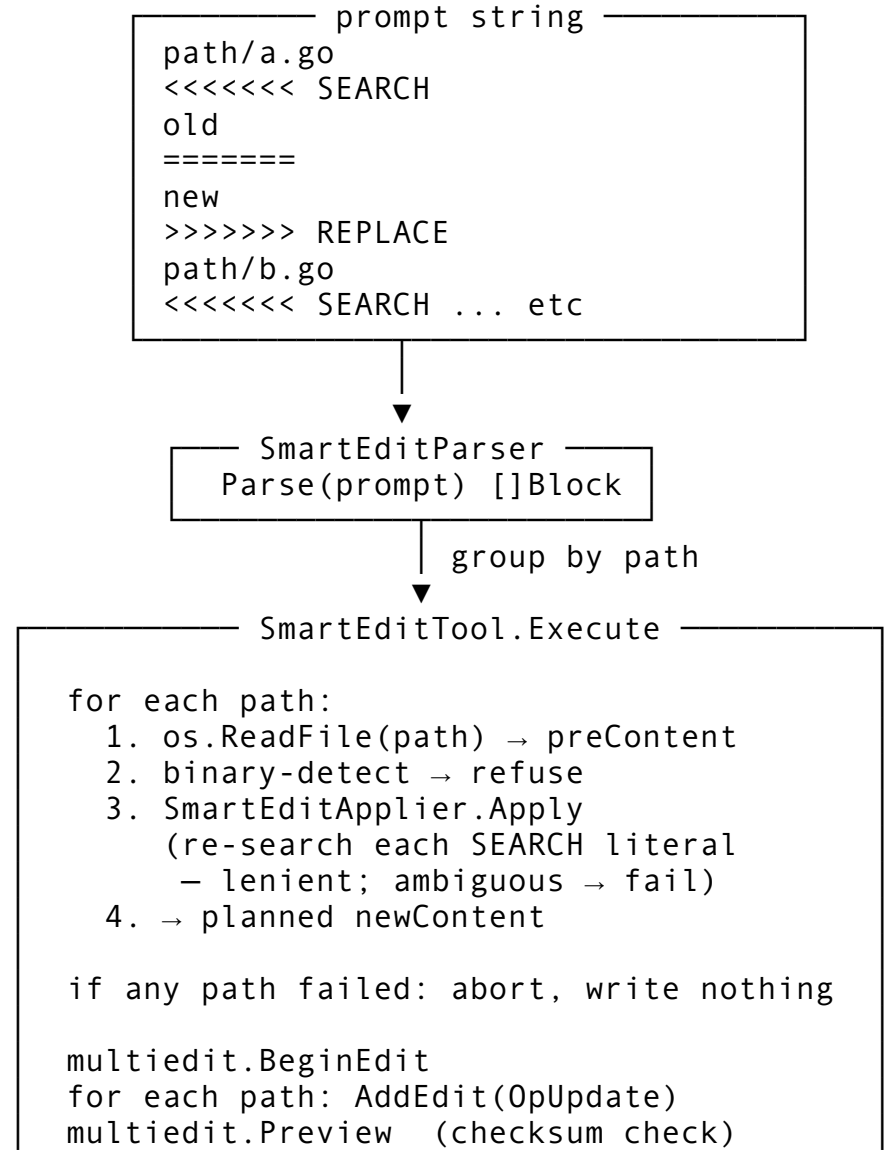
Four layers, all under `helix_code/internal/tools/smartedit/`, plus a slash command under `helix_code/internal/commands/`:

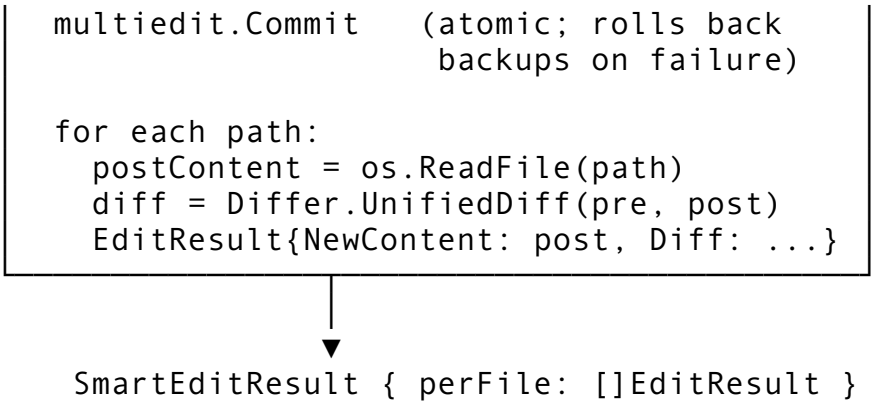
- `SmartEditParser` (`parser.go`) — pure function `Parse(prompt string) ([]EditBlock, error)`. Tokenises a prompt string into `EditBlock` records by scanning for the delimiter triplet `<<<<<<< SEARCH` / `=======` / `>>>>>>> REPLACE`. Each block carries its file path (the line immediately preceding `<<<<<<< SEARCH`), the SEARCH literal, and the REPLACE literal. No filesystem access; no stateful side effects.
- `SmartEditApplier` (`applier.go`) — pure function `Apply(content string, blocks []EditBlock) (string, []BlockResult, error)`. Takes a file's current content + the list of blocks targeting that file; for each block, locates the SEARCH literal in the current content, ensures it is present **exactly once** (lenient conflict detection per Q4=B; ambiguous → fail), and replaces it with REPLACE. Returns the new content, per-block success/failure, and an aggregate error (the first block failure aborts the file). No filesystem access.
- `Differ` (`diff.go`) — pure function `UnifiedDiff(oldContent, newContent, path string) string`. Pure-Go, line-by-line; uses the LCS-based diff algorithm already shipped in `internal/tools/multiedit/diff.go::DiffManager.GenerateDiff` (re-used, not re-implemented; see §3.6). Returns a unified-diff `string` (not a struct) for direct printing to the caller; the per-file `EditResult.Diff` field is a `string` so the agent can grep / pattern-match it without further unmarshaling.
- `SmartEditTool` (`smart_edit_tool.go`) — `Tool` interface impl that wires the above three together with the multiedit `MultiFileEditor`. On `Execute(ctx, params)`:
  1. Parse `params["prompt"]` → `[]EditBlock`.
  2. Group blocks by file path → `map[string][]EditBlock`.
  3. For each file: read current content via `os.ReadFile` (real disk; no fs abstraction so binary-detect runs against the actual bytes); detect binary; refuse binary; run `SmartEditApplier.Apply` on the content; if any block fails, abort the whole

prompt with `ErrApplyFailed` and return without writing. Capture pre-content + planned post-content per file.

4. If all files succeeded in-memory: open a multiedit transaction (`MultiFileEditor.BeginEdit`), add one `FileEdit{Operation: OpUpdate, OldContent, NewContent}` per file, call `Preview` (drives the existing checksum pipeline; checksums for the lenient model are computed from the pre-content we just read so the multiedit checksum check trivially passes), then `Commit`. Multiedit's `Commit` is atomic + rolled back on per-file write failure (existing F08 behaviour).
5. After `Commit` returns success: re-read each file from disk (`os.ReadFile`), compute `UnifiedDiff(preContent, postReadContent, path)`, populate `EditResult.NewContent` (post-read) + `EditResult.Diff`. **The re-read is mandatory** — anti-bluff for "file unchanged after success" (§5.2).

- `/edit` **slash command** (`internal/commands/edit_command.go`) — mirrors F14 `/sandbox` and F16 `/telemetry` shape: defines a `SmartEditInspector` interface in the commands package so the slash is testable with a fake while main.go passes the real `*smartedit.SmartEditTool`. Subcommands: `status` (last attempt's per-file pass/fail summary), `diff` (last attempt's diffs concatenated), `dry-run <path>` (parse blocks from `<path>`, apply in-memory, return diff WITHOUT writing), `commit <path>` (parse + apply + write; equivalent to invoking the tool directly with the file's contents as `prompt`).





Why a tool + slash + no cobra subcommand: - `smart_edit` **tool** — primary surface for the LLM. The agent emits a tool call; the registry dispatches. Same lifecycle as `fs_edit` / `multiedit_commit`. - `/edit` **slash** — primary surface for the human in the TUI. `dry-run` is the killer feature — load blocks from a scratch file, see the diff, decide. - **No cobra subcommand** (Q5=A) — `helixcode edit ...` would duplicate the slash without adding value; the file-load path of `/edit dry-run <path>` already covers scripted use via `helixcode --slash-command "/edit dry-run blocks.txt"`.

---

3. Components

3.1 New files

- `helix_code/internal/tools/smartedit/types.go` — `EditBlock`, `EditPlan`, `BlockResult`, `EditResult`, `SmartEditResult`, marker constants (`searchMarker`, `dividerMarker`, `replaceMarker`), error sentinels.
- `helix_code/internal/tools/smartedit/types_test.go`.
- `helix_code/internal/tools/smartedit/parser.go` — `Parse(prompt string) ([]EditBlock, error)`, marker scanner.
- `helix_code/internal/tools/smartedit/parser_test.go`.
- `helix_code/internal/tools/smartedit/applier.go` — `Apply(content string, blocks []EditBlock) (string, []BlockResult, error)`; `findExactlyOnce(content, search string) (int, error)`.
- `helix_code/internal/tools/smartedit/applier_test.go`.
- `helix_code/internal/tools/smartedit/diff.go` — `UnifiedDiff(oldContent, newContent, path string) string`; thin wrapper around `multiedit.DiffManager.GenerateDiff` (re-uses existing F08 LCS implementation). Lives in this package so the smart-edit tool does not pull every multiedit symbol into its public API.
- `helix_code/internal/tools/smartedit/diff_test.go`.
- `helix_code/internal/tools/smartedit/smart_edit_tool.go` — `SmartEditTool` (Tool impl); embeds `*multiedit.MultiFileEditor`; tracks last attempt for `/edit status` / `/edit diff`.
- `helix_code/internal/tools/smartedit/smart_edit_tool_test.go`.
- `helix_code/internal/tools/smartedit/binary_detect.go` — `IsBinary(content []byte) bool` per the standard "first 8KiB contains a NUL byte" heuristic. Lives in this package (not in filesystem) so the binary-refusal contract is local to smart-edit; if filesystem grows its own binary-detect later they can de-duplicate.
- `helix_code/internal/tools/smartedit/binary_detect_test.go`.

- `helix_code/internal/commands/edit_command.go` — `/edit` slash (`status` / `diff` / `dry-run` / `commit`).
- `helix_code/internal/commands/edit_command_test.go`.
- `helix_code/tests/integration/smartedit_test.go` — `//go:build integration`. Real tempdir, real disk writes, full pipeline.
- `helix_code/tests/integration/cmd/p1f17_challenge/main.go` — runtime evidence harness.
- `challenges/p1-f17-smart-file-editing/CHALLENGE.md` + `run.sh`.

3.2 Modified files

- `helix_code/cmd/cli/main.go` — three lines: construct `SmartEditTool` (passes the existing `*multiedit.MultiFileEditor` already wired by F08), register into the tool registry, register `/edit` slash.
- `helix_code/internal/tools/registry.go` — register `smart_edit` in the `Tool` map alongside `fs_edit` / `multiedit_commit` (single line in the existing init block; no new fields, no new setters — `SmartEditTool` is opaque to the registry).

No new external dependencies (§3.5).

3.3 Types

```
// internal/tools/smartedit/types.go

const (
    searchMarker   = "<<<<<<< SEARCH"
    dividerMarker  = "======="
    replaceMarker  = ">>>>>>> REPLACE"
)

// EditBlock is a single SEARCH/REPLACE pair targeting a specific
        file.
type EditBlock struct {
    Path    string // relative or absolute file path (line
        preceding the SEARCH marker)
    Search  string // literal text to find in the file
    Replace string // literal text to substitute
    Line
        int    // 1-based line number of the SEARCH marker in the
        prompt (for error messages)
}

// EditPlan groups blocks by file. Convenience type used
        internally.
type EditPlan map[string][]EditBlock

// BlockResult records the outcome of a single block.
type BlockResult struct {
    Block   EditBlock
    Applied bool
```

```
    Reason  string // populated on failure: "search not found",
        "ambiguous match", ...
}

// EditResult is the per-file outcome of a smart_edit Execute
        call.
type EditResult struct {
    Path        string
    Applied     bool          // true iff every block targeting
        this file applied AND the file was rewritten on disk
    NewContent  string        // post-write re-read from disk
        (NOT the in-memory plan); empty when Applied=false
    Diff        string        // unified diff between pre-apply
        and post-write content; empty when Applied=false
    Blocks      []BlockResult // one entry per block; reflects
        in-memory apply (matches the disk state when
        Applied=true)
    Error       error         // top-level error for this file
        (binary refusal, search-not-found, ambiguous, ...)
}

// SmartEditResult is the SmartEditTool.Execute return shape.
type SmartEditResult struct {
    Files      []EditResult
    Applied
        bool          // true iff every file in the prompt
        applied successfully
    Diff       string        // concatenated diffs across all
        files (empty when Applied=false)
    DurationMs int64
}

// Error sentinels — wrap with %w; tests use errors.Is.
var (
    ErrParseEmpty         = errors.New("smartedit: prompt
        contains no edit blocks")
    ErrParseMalformed     =
        errors.New("smartedit: malformed edit block (missing
        marker)")
    ErrParseNoPath        = errors.New("smartedit: SEARCH marker
        without preceding file path")
    ErrSearchNotFound     =
        errors.New("smartedit: search pattern not found in file")
    ErrSearchAmbiguous    =
        errors.New("smartedit: search pattern matches multiple
        times")
    ErrEmptySearch        = errors.New("smartedit: SEARCH block
        is empty")
```

```
    ErrBinaryFile         = errors.New("smartedit: target file
        appears to be binary; refusing to edit")
    ErrApplyFailed        = errors.New("smartedit: one or more
        blocks failed; nothing written")
    ErrFileTooLarge       = errors.New("smartedit: file exceeds
        size limit")
    ErrPromptTooLarge     =
        errors.New("smartedit: prompt exceeds size limit")
)

// SizeLimits guard against pathological prompts.
const (
    MaxPromptBytes = 4 * 1024 * 1024  // 4 MiB; covers a multi-
        file refactor of a few hundred files
    MaxFileBytes   = 4 * 1024 * 1024  // mirrors
        multiedit.DefaultConfig().MaxFileSize / 2.5
)

// internal/tools/smartedit/parser.go

// Parse tokenises a prompt string into edit blocks. Lines
        preceding a
// `<<<<<<< SEARCH` marker line are treated as the file path
        target.
//
// Format:
//
//     path/to/file.ext
//     <<<<<<< SEARCH
//     <old>
//     =======
//     <new>
//     >>>>>>> REPLACE
//
// Multiple blocks per file and multiple files per prompt are
        supported.
// Path lines are stripped of leading/trailing whitespace.
func Parse(prompt string) ([]EditBlock, error)

// internal/tools/smartedit/applier.go

// Apply applies blocks to content in order. Each block's SEARCH
        must
// appear EXACTLY ONCE in the current (mutated) content. If
        absent →
// ErrSearchNotFound. If present multiple times →
        ErrSearchAmbiguous.
// On the first block-level failure, returns (currentContent,
        results, err)
```

```
// — currentContent reflects partial in-memory state; the caller
        MUST NOT
// write it. The SmartEditTool only writes when the aggregate err
        is nil
// for every targeted file.
func Apply(content string, blocks []EditBlock) (string,
        []BlockResult, error)

// findExactlyOnce returns the byte offset of the unique
        occurrence of
// search in content, or an error if absent / ambiguous. Empty
        search →
// ErrEmptySearch.
func findExactlyOnce(content, search string) (int, error)

// internal/tools/smartedit/diff.go

// UnifiedDiff returns a unified-format diff between oldContent
        and
// newContent for the given path. Wraps the existing F08
// multiedit.DiffManager.GenerateDiff to keep a single LCS
        implementation
// in the codebase.
func UnifiedDiff(oldContent, newContent, path string) string

// internal/tools/smartedit/smart_edit_tool.go

// SmartEditTool is the Tool-interface implementation registered
        as
// "smart_edit". It owns the multiedit transaction lifecycle for
        the
// blocks parsed from a single Execute call.
type SmartEditTool struct {
    multiEdit *multiedit.MultiFileEditor // F08 transactional
        editor
    logger    *zap.Logger

    mu          sync.RWMutex
    lastResult  *SmartEditResult // populated on every Execute
        (success or failure)
    lastBlocks  []EditBlock      // populated on every Execute
        (post-Parse)
}

func NewSmartEditTool(me *multiedit.MultiFileEditor, logger
        *zap.Logger) *SmartEditTool

// Tool interface
```

```
func (t *SmartEditTool) Name() string                          {
        return "smart_edit" }
func (t *SmartEditTool) Description() string                   {
        return "Apply SEARCH/REPLACE blocks to one or more files
        atomically." }
func (t *SmartEditTool) Schema() tools.ToolSchema
func (t *SmartEditTool) Category() tools.ToolCategory          {
        return tools.CategoryMultiEdit }
func (t *SmartEditTool) Validate(params map[string]interface{})
        error
func (t *SmartEditTool) Execute(ctx context.Context, params
        map[string]interface{}) (interface{}, error)

// SmartEditInspector — interface used by /edit slash; matches /
        sandbox + /telemetry shape.
type SmartEditInspector interface {
    LastResult() *SmartEditResult
    LastBlocks() []EditBlock
    DryRun(ctx context.Context, prompt string)
        (*SmartEditResult, error) // no-write
    Commit(ctx context.Context, prompt string)
        (*SmartEditResult, error) // writes
}

func (t *SmartEditTool) LastResult() *SmartEditResult
func (t *SmartEditTool) LastBlocks() []EditBlock
func (t *SmartEditTool) DryRun(ctx context.Context, prompt
        string) (*SmartEditResult, error)
func (t *SmartEditTool) Commit(ctx context.Context, prompt
        string) (*SmartEditResult, error)

// internal/commands/edit_command.go

type EditCommand struct {
    inspector smartedit.SmartEditInspector
}

func NewEditCommand(insp smartedit.SmartEditInspector)
        *EditCommand

func (c *EditCommand) Name() string         { return "edit" }
func (c *EditCommand) Aliases() []string    { return nil }
func (c *EditCommand) Description() string  { return "Inspect
        last smart_edit attempt or run a dry-run / commit from a
        blocks file." }
func (c *EditCommand) Usage() string        { return "/edit
        [status|diff|dry-run <path>|commit <path>]" }
func (c *EditCommand) Execute(ctx context.Context, cc
        *CommandContext) (*CommandResult, error)
```

3.4 User surfaces

`smart_edit` **tool** — single parameter `prompt`:

```
{
  "name": "smart_edit",
  "parameters": {
    "prompt": "path/to/file.go\n<<<<<<<
        SEARCH\nfoo\n=======\nbar\n>>>>>>> REPLACE\n"
  }
}
```

Returns `SmartEditResult`. On `Applied=false`, `Diff` is empty and per-file `EditResult` carry the reason. The agent is expected to inspect `result.Files[i].Diff` to confirm the change actually landed; an empty diff with `Applied=true` is a contradiction the test suite catches (§5.2).

`/edit` **slash** — TUI inspector:

Subcommand	Behaviour
`/edit` (alias `/edit status`)	`STATUS PATH BLOCKS +ADDED -REMOVED` table for the last attempt. When no attempt yet → `no smart_edit attempts in this session.`
`/edit diff`	Concatenated unified diffs from the last attempt (only files that applied). When the last attempt failed → reports per-file failure reasons.
`/edit dry-run <path>`	Reads `<path>` (a file containing one or more SEARCH/REPLACE blocks), parses, applies in-memory, returns the diff. **No disk writes.**
`/edit commit <path>`	Reads `<path>`, parses, applies, writes. Equivalent to invoking `smart_edit` directly with the file's contents.

`/edit diff` output **MUST** be visible to the user but **MUST NOT** be logged at INFO level by the host process — see §5.2 (CONST-042). The diff is printed to the slash command's response (which is a user-facing surface; if the user is sharing terminal screenshots that's their decision); zap loggers only see a one-line `INFO smart_edit applied (n files, m bytes diff)` summary. The full diff text is logged at DEBUG only.

No cobra subcommand (Q5=A). Inspection and dry-run/commit both go through the slash.

3.5 New external dependencies

None. F17 is built entirely on the Go standard library (`os`, `strings`, `bufio`, `errors`, `fmt`, `sync`, `time`) plus the existing testify (v1.11) and zap (already in go.mod via every other feature). The diff is computed by re-using `internal/tools/multiedit/`

`diff.go::DiffManager.GenerateDiff` (already in go.mod since F08). Brief justification:

- Search-replace block parsing is line-by-line text — `bufio.Scanner` on a string suffices.
- Apply is `strings.Index` + `strings.Count` + a single `strings.Replace` per block.
- Diff is delegated to the existing F08 `DiffManager` (LCS-based unified diff already in production) — re-use is the explicit anti-bluff choice (one diff implementation, one set of tests).
- Atomicity is delegated to F08 `MultiFileEditor` (already in production) — re-use, not re-implement.

`go mod tidy` after T02 must produce **zero new entries in** `go.sum`. If it doesn't, that's a red flag the implementation drifted from the spec.

3.6 Existing-code constraints

- `internal/tools/multiedit/multiedit.go::MultiFileEditor.BeginEdit / AddEdit / Preview / Commit / Rollback` — already transactional. F17 calls them in-order. The transaction's `EditOptions.BackupEnabled` is left at the multiedit default (true) so any per-file write failure during `Commit` triggers backup-restore for the files that already wrote — the all-or-nothing guarantee (Q3=B) inherits from F08.
- `internal/tools/multiedit/transaction.go::FileEdit{Operation: OpUpdate, OldContent, NewContent, Checksum}` — F17 populates `OldContent` with the pre-read bytes and `Checksum` with `calculateChecksum(OldContent)`. Because we just read the content moments earlier, the checksum check in `verifyChecksum` (line 489) is trivially satisfied for the lenient model. We do NOT use multiedit's stricter conflict policies (`ConflictPolicyAbort` etc.) for SEARCH-not-found — that's our own lenient-model concern handled in the applier before we ever call `BeginEdit`.
- `internal/tools/multiedit/diff.go::DiffManager.GenerateDiff(old, new []byte, path string) (*Diff, error)` — exposes `Diff.Unified` (a `string`). `smartedit.UnifiedDiff` calls this and returns `Diff.Unified`. The wrapping is one function; the choice to keep a `string` return (not the full `*Diff` struct) is deliberate (§11 below).
- `internal/tools/registry.go::ToolRegistry` — F17 registers `smart_edit` exactly like F09's `multiedit_commit`. No registry interface changes; no new setter; no new field.
- `internal/commands/registry.go` — F17 registers `EditCommand` exactly like F14 `/sandbox` and F16 `/telemetry`. No new registration mechanics.
- `cmd/cli/main.go` — F17 wires alongside the existing `multiedit.NewMultiFileEditor(...)` call in `main()`. Three new lines: `smartTool := smartedit.NewSmartEditTool(mfe, logger)`, `toolReg.Register(smartTool)`, `slashRegistry.Register(commands.NewEditCommand(smartTool))`.

4. Data flow

4.1 SmartEditTool.Execute

```
SmartEditTool.Execute(ctx, params)
  ├─ start := time.Now()
  ├─ prompt, ok := params["prompt"].(string); if !ok || prompt == "" → Err
  ├─ if len(prompt) > MaxPromptBytes → ErrPromptTooLarge
  ├─ blocks, err := Parse(prompt); if err → return SmartEditResult{Applied
  ├─ if len(blocks) == 0 → ErrParseEmpty
  │
  ├─ plan := EditPlan{}
  ├─ for _, b := range blocks: plan[b.Path] = append(plan[b.Path], b)
  │
  ├─ // Phase 1: in-memory apply, no disk writes yet
  ├─ planned := map[string]planEntry{}  // path → {pre, post, blockResults
  ├─ for path, fileBlocks := range plan:
  │     pre, err := os.ReadFile(path); if err → record fail; continue
  │     if len(pre) > MaxFileBytes → record fail (ErrFileTooLarge); contin
  │     if IsBinary(pre) → record fail (ErrBinaryFile); continue
  │     post, blockResults, applyErr := Apply(string(pre), fileBlocks)
  │     if applyErr → record fail; continue
  │     planned[path] = {pre: pre, post: []byte(post), blockResults: block
  │
  ├─ if any path failed → return SmartEditResult{Applied:false, Files: ...
  │     // NO writes; Q3=B all-or-nothing
  │
  ├─ // Phase 2: transactional commit through multiedit
  ├─ tx, err := t.multiEdit.BeginEdit(ctx, multiedit.EditOptions{...}); if
  ├─ for path, entry := range planned:
  │     fe := &multiedit.FileEdit{
  │         FilePath:   path,
  │         Operation:  multiedit.OpUpdate,
  │         OldContent: entry.pre,
  │         NewContent: entry.post,
  │         Checksum:   sha256(entry.pre),  // lenient: trivially satisfie
  │     }
  │     if err := t.multiEdit.AddEdit(ctx, tx, fe); err → fail; rollback
  │
  ├─ if _, err := t.multiEdit.Preview(ctx, tx); err → fail; rollback
  ├─ if err := t.multiEdit.Commit(ctx, tx); err →
  │     // multiedit already rolled back internally
  │     return SmartEditResult{Applied:false}, fmt.Errorf("commit failed:
  │
  ├─ // Phase 3: re-read each file from disk; compute diff
  ├─ result := SmartEditResult{Applied: true, DurationMs: ms}
  ├─ for path, entry := range planned:
  │     postRead, err := os.ReadFile(path); if err → mark file failed (rar
  │     diff := UnifiedDiff(string(entry.pre), string(postRead), path)
  │     result.Files = append(result.Files, EditResult{
  │         Path: path, Applied: true,
  │         NewContent: string(postRead),
  │         Diff: diff,
```

```
│          Blocks: entry.blockResults,
│      })
│      result.Diff += diff + "\n"
│
├─ t.mu.Lock(); t.lastResult = &result; t.lastBlocks = blocks; t.mu.Unlo
├─ logger.Info("smart_edit applied", zap.Int("files", len(result.Files))
├─ logger.Debug("smart_edit diff", zap.String("diff", result.Diff))   //
└─ return result, nil
```

4.2 Parser data flow

The parser is a state machine over the prompt's lines:

```
state ∈ {scanPath, scanSearch, scanReplace}
pendingPath := ""
buf := []string

for each line in prompt (1-indexed):
  switch state:
    scanPath:
      if line == searchMarker:
        if pendingPath == "":
          return nil, ErrParseNoPath
        state = scanSearch; buf = nil
      else if !isBlankOrComment(line):
        pendingPath = strings.TrimSpace(line)
    scanSearch:
      if line == dividerMarker:
        searchText = strings.Join(buf, "\n")
        state = scanReplace; buf = nil
      else:
        buf = append(buf, line)
    scanReplace:
      if line == replaceMarker:
        replaceText = strings.Join(buf, "\n")
        emit EditBlock{Path:pendingPath, Search:searchText, Replace:replac
        state = scanPath; buf = nil
        // pendingPath retained for next block targeting same file (a bare
        // <<<<<<< SEARCH with no path line in between targets the same fi
      else:
        buf = append(buf, line)

if state != scanPath: ErrParseMalformed
```

Path-line stickiness: within a prompt, the path stays attached to subsequent blocks until a new path line is encountered. This means consecutive blocks for the same file can omit repeated path lines — common in the claude-code/aider conventions.

4.3 Applier data flow

```
Apply(content, blocks):
  current := content
  results := []BlockResult{}
```

```
  for _, b := range blocks:
    if b.Search == "":
      results = append(results, BlockResult{Block:b, Applied:false, Reason
      return current, results, ErrEmptySearch
    n := strings.Count(current, b.Search)
    switch n {
    case 0:
      results = append(results, BlockResult{Block:b, Applied:false, Reason
      return current, results, ErrSearchNotFound
    case 1:
      current = strings.Replace(current, b.Search, b.Replace, 1)
      results = append(results, BlockResult{Block:b, Applied:true})
    default:
      results = append(results, BlockResult{Block:b, Applied:false, Reason
      return current, results, ErrSearchAmbiguous
    }
  return current, results, nil
```

Block ordering matters: a later block's SEARCH may be the result of an earlier block's REPLACE. The applier processes blocks in the order they appeared in the prompt. This matches claude-code/aider behaviour and is what the LLM expects.

4.4 /edit slash flow

```
EditCommand.Execute(ctx, cc):
  if c.inspector == nil: return CommandResult{Success:false, Output:"smart
  sub := "status"
  if len(cc.Args) > 0: sub = cc.Args[0]
  switch sub {
    case "status":   return c.handleStatus(), nil
    case "diff":     return c.handleDiff(), nil
    case "dry-run":  if len(cc.Args)<2: usage; return c.handleDryRun(ctx,
    case "commit":   if len(cc.Args)<2: usage; return c.handleCommit(ctx,
    default:         return CommandResult{Success:false, Output:c.Usage()}
  }
```

`handleDryRun(path)`: `os.ReadFile(path)` → `prompt`; `inspector.DryRun(ctx, prompt)`; format result; **never writes**. Implemented by `SmartEditTool.DryRun` which runs Phase 1 only (parse + in-memory apply + diff against the planned post-content) and returns without invoking multiedit.

`handleCommit(path)`: `os.ReadFile(path)` → `prompt`; `inspector.Commit(ctx, prompt)`; format result; equivalent to invoking the `smart_edit` tool with the file's contents.

4.5 Single-file vs multi-file path

Single-file is a degenerate multi-file: one transaction, one `FileEdit`, same atomicity. There is no separate code path. The test suite includes a deliberate single-file Challenge phase to verify the degenerate case is honoured (§6.3 phase 1).

5. Error handling, edge cases, and anti-bluff

5.1 Error paths

- **Empty prompt** — `ErrParseEmpty` from `Parse`. Tool returns it with `Applied:false`.
- **Missing path line** (a `<<<<<<< SEARCH` with no preceding non-blank line) — `ErrParseNoPath`. The block's line number is in the message.
- **Missing `=======` divider** — parser stays in `scanSearch` past EOF → `ErrParseMalformed`.
- **Missing `>>>>>>> REPLACE` terminator** — same shape; `ErrParseMalformed`.
- **Empty SEARCH** — `ErrEmptySearch` from `Apply`. Refusing prevents an unbounded `strings.Replace("", "")` substitution that would degenerate to a no-op everywhere.
- **SEARCH not found in file** — `ErrSearchNotFound`. The lenient model: re-search at apply time, fail this block, abort the whole prompt (Q3=B), do NOT write anything. The error message includes the path and the first 80 chars of SEARCH for debuggability.
- **SEARCH appears multiple times** — `ErrSearchAmbiguous`. The LLM is expected to disambiguate by widening the SEARCH to include surrounding context. The error message includes the count.
- **Binary file** — `ErrBinaryFile`. Detected by NUL byte in first 8 KiB (standard heuristic). Refused without reading further; never written.
- **File too large** — `ErrFileTooLarge` (default 4 MiB; matches multiedit's per-file cap when halved). Larger files require explicit override (not in v1).
- **Prompt too large** — `ErrPromptTooLarge` (4 MiB). A multi-hundred-file refactor still fits; pathological prompts don't.
- **Multiedit `BeginEdit` / `AddEdit` / `Preview` / `Commit` failure** — propagated with `%w`; multiedit handles its own rollback. The tool's `Applied` is `false`; nothing on disk changed.
- **Re-read after Commit fails** — extremely rare (multiedit just wrote the file). Marked as a per-file failure; the file *probably* did get written but we can't confirm; the test suite asserts re-read errors are surfaced loudly rather than swallowed.
- **External modification between `os.ReadFile` and multiedit `Commit`** — small race window. Multiedit's `verifyChecksum` will fail with `ConflictError` (lines 489–510). The smart-edit tool propagates this as `Applied:false`. Documented; not closed in v1 (would require a multi-edit-internal lock; deferred).

5.2 Anti-bluff (CONST-035 / §11.9) — LOUD

The single largest bluff vector for F17 is "tool says edit applied but file unchanged on disk." This compiles, passes naive unit tests, and silently corrupts the agent's mental model of the codebase. Common bluff variants:

1. **(a) `Applied=true` but file unchanged** — Apply ran in-memory; the multiedit Commit failed silently or was never called; the tool returned `Applied=true` because the in-memory plan succeeded. **Defence**: the post-Commit re-read is mandatory; `EditResult.NewContent` is always populated from disk, never from the planned post-content; a unit test deliberately makes the multiedit Commit fail and asserts `Applied=false`.
2. **(b) SEARCH not found, but tool returns success** — the applier silently skipped the block, or the regex/index was off-by-one and matched the empty string. **Defence**: `Apply` returns `ErrSearchNotFound` on `strings.Count == 0`; the tool's Phase-1 abort

runs before any write; a unit test passes a SEARCH that's literally not in the file and asserts the tool returns an error AND the file is byte-identical after the call.

3. **(c) Multi-file commit reports success but only some files were written** — a write failure on file 3 of 5 leaves files 1–2 written and 3–5 not (partial commit; atomicity broken). **Defence**: F17 delegates to multiedit's `Commit` which uses backups + reverse-order restore on per-file failure; an integration test deliberately inserts a write-permission error on file 3 and asserts ALL FIVE FILES ARE BYTE-IDENTICAL TO THEIR ORIGINAL CONTENT (i.e., the rollback worked). The Challenge harness's "PARTIAL ROLLBACK" phase is the canonical assertion.
4. **(d) Diff returned doesn't match what's actually on disk** — the diff was computed from the planned post-content, not the re-read post-write content. **Defence**: `UnifiedDiff` is invoked with `(preContent, postReadFromDisk, path)` — never with the planned content. A test feeds the tool a known SEARCH/REPLACE, then independently runs `diff -u` (via `os/exec`) on the before/after files captured by the test harness, and asserts the tool's `Diff` is byte-identical to the `diff -u` output (modulo whitespace canonicalisation).

Required real-execution criteria (these define what "smart edit works" means in F17):

1. **Unit tests** — real tempdirs (`t.TempDir()`), real `os.WriteFile` / `os.ReadFile`. **NO mocked filesystem.** The applier and parser have pure-function tests against in-memory strings (allowed); the tool's tests always go through the disk.
2. **Integration tests** (`-tags=integration`) — exercise the full pipeline (Parse → Apply → multiedit transaction → re-read → diff) against a real tempdir with multiple files. ALWAYS-runs (no infrastructure dependency). Includes:
   - Single-file edit success → file content actually changed.
   - SEARCH-not-found rejection → file byte-identical to original.
   - Multi-file atomic commit → all N files have the planned post-content on disk.
   - Multi-file partial-failure rollback → injects a permission error mid-commit; asserts ALL N files are byte-identical to their pre-content.
   - Diff exactness → tool's `Diff` matches independent `diff -u` output.
   - Ambiguous SEARCH rejection → file byte-identical.
   - Binary file refusal → file byte-identical.
3. **Challenge harness** — the canonical CONST-039 evidence script. Phases (§6.3): SINGLE-FILE-SUCCESS, SEARCH-NOT-FOUND-REJECTED, MULTI-FILE-ATOMIC, PARTIAL-FAILURE-ROLLBACK, DIFF-EXACTNESS, AMBIGUOUS-REJECTED, BINARY-REFUSED. Every phase asserts disk state with positive evidence — a `sha256sum` of the file before and after is captured into the harness output.
4. **Challenge MUST exit non-zero on any disk-state mismatch.** "tool returned success but file is wrong" is a hard failure; absence-of-error is NEVER acceptable.

Concrete forbidden phrases (anti-bluff smoke):

```
cd HelixCode && grep -rn "simulated\|for now\|TODO implement\|
        placeholder" \
  internal/tools/smartedit internal/commands/edit_command.go \
  && echo BLUFF || echo clean
```

Must always print `clean`.

CONST-042 secret-content protection (mandatory):

File contents may include secrets (the user is editing source files; an `.env` accidentally edited would have `API_KEY=sk-…` lines in both SEARCH and REPLACE). The mechanism:

- **No `Diff` text at INFO level.** The host process logs `INFO smart_edit applied (n files, m bytes diff)` only — bytes count, no text. Full diff text is logged at DEBUG only. A unit test runs the tool with `zaptest.NewLogger(t, zaptest.Level(zapcore.InfoLevel))`, edits a file with the literal string `SECRET_TOKEN=sk-deadbeef`, captures the log buffer, and asserts the buffer does NOT contain `sk-deadbeef`.
- **Diff returned to caller IS visible** — the user invoked the tool; they get to see what changed. This is by design. The slash command's `/edit diff` output is user-facing, not log output.
- **No `Diff` text in the Challenge's saved evidence file** — the Challenge harness records SHA-256 hashes + line counts of diffs, not the diff text itself, into `06_phase_1_evidence.md`. The harness's stdout (which captures sample diffs for human inspection) is treated as transient and not committed.
- **Hardcoded blocklist** — F17 does NOT blocklist content patterns inside SEARCH/REPLACE (the user is editing files; we cannot second-guess what they're editing). The protection is at the logging layer, not the tool boundary.

Real-execution criteria summary:

Criterion	Unit	Integration
(1) `Applied=true` ⇒ file changed on disk	`TestSmartEditTool_AppliedTrue_ImpliesFileChanged` (sha before/after)	`TestSmart`
(2) `Applied=false` ⇒ file unchanged on disk	`TestSmartEditTool_AppliedFalse_FileUnchanged`	`TestSmart`
(3) Multi-file atomic	`TestApplier_MultiBlock_OrderRespected`	`TestSmart`
(4) Partial-failure rollback	n/a (needs disk)	`TestSmart`
(5) Diff exactness	`TestUnifiedDiff_MatchesDiffU`	`TestSmart`
(6) Ambiguous rejected	`TestApplier_Ambiguous_Returns_ErrSearchAmbiguous`	`TestSmart`
(7) Binary refused	`TestIsBinary_DetectsNUL`	`TestSmart`

Criterion	Unit	Integration
(8) No diff text at INFO	`TestSmartEditTool_DoesNotLogDiffAtInfo`	n/a

The Challenge harness uses positive evidence: `if sha256_post == sha256_pre AND tool_said_applied: exit 1`. Absence-of-error is NEVER acceptable — a Challenge that reports PASS without observed positive evidence is itself a bluff.

5.3 Edge case: SEARCH block that contains the marker triplet itself

If the SEARCH or REPLACE text needs to literally contain `<<<<<<< SEARCH` (e.g., editing a doc *about* the smart-edit format — yes, this very spec), the parser would mis-tokenise the inner marker as a block boundary.

v1 resolution: the parser is **strict literal** — a line that starts with `<<<<<<< SEARCH` (after `TrimRight(" \t")`) is always a marker. There is **no escape mechanism in v1**. To edit a file that contains the markers, the user must:

1. Use `fs_write` (overwrite the file with the new content directly), OR
2. Avoid putting the marker literal on a line that starts at column 0 by indenting it (the parser only treats column-0 markers as boundaries).

This is the same constraint claude-code and aider apply. Adding an escape mechanism (e.g., `\<<<<<<< SEARCH` at column 0 to treat as literal) is **deferred to v2** because:

- Real-world frequency is essentially zero outside spec / docs / tests of the smart-edit format itself.
- Any escape mechanism we pick now risks colliding with claude-code/aider conventions.
- The workaround (indent the literal, or use `fs_write`) is one keystroke.

A unit test (`TestParser_RejectsCol0MarkerInsideBlock_DocumentingLimitation`) explicitly documents the limitation: a SEARCH block whose body contains `<<<<<<< SEARCH` at column 0 will be parsed as a malformed prompt (the inner marker terminates the outer SEARCH prematurely; the divider check then fails). The test pins the failure mode so a future v2 can introduce escaping deliberately.

For *this spec file's* meta-references to the markers (which the spec inevitably contains), the convention is to indent every example marker by two spaces inside fenced code blocks — both this spec and the plan honour this.

6. Testing

6.1 Unit (mocks not used; real tempdirs for tool tests; pure-function tests for parser/applier)

Parser (`parser_test.go`): - `TestParse_SingleBlock_OK`. - `TestParse_MultipleBlocksSameFile_OK` (path-line stickiness). - `TestParse_MultipleFiles_OK`. - `TestParse_EmptyPrompt_ErrParseEmpty`. - `TestParse_NoBlocks_ErrParseEmpty`. - `TestParse_SearchWithoutPath_ErrParseNoPath`. - `TestParse_MissingDivider_ErrParseMalformed`. -

TestParse_MissingTerminator_ErrParseMalformed. - TestParse_BlankLinesBeforePath_OK. - TestParse_LineNumberRecordedInBlock_OK. - TestParse_RejectsCol0MarkerInsideBlock_DocumentingLimitation.

Applier (applier_test.go): - TestApply_SingleBlock_Replaces. - TestApply_SearchNotFound_ErrSearchNotFound. - TestApply_AmbiguousSearch_ErrSearchAmbiguous. - TestApply_EmptySearch_ErrEmptySearch. - TestApply_OrderRespected_LaterBlockSeesEarlierReplace. - TestApply_FirstFailureAborts_LaterBlocksUnattempted. - TestApply_MultilineSearchReplace_OK. - TestApply_PreservesContentOutsideMatch.

Differ (diff_test.go): - TestUnifiedDiff_MatchesDiffU — runs the system diff -u on the same input via os/exec (skipped if diff not installed; SKIP-OK marker) and asserts byte-equal modulo trailing-newline canonicalisation. - TestUnifiedDiff_EmptyOldFile_AllAdds. - TestUnifiedDiff_EmptyNewFile_AllRemoves. - TestUnifiedDiff_NoChange_EmptyDiff.

Binary detect (binary_detect_test.go): - TestIsBinary_DetectsNUL. - TestIsBinary_TextFile_False. - TestIsBinary_EmptyFile_False. - TestIsBinary_FirstByteIsNUL_True.

Tool (smart_edit_tool_test.go) — these use real tempdirs: - TestSmartEditTool_AppliedTrue_ImpliesFileChanged (sha-before vs sha-after). - TestSmartEditTool_AppliedFalse_FileUnchanged (deliberately unfindable SEARCH; sha-before == sha-after). - TestSmartEditTool_SingleFile_NewContentMatchesDisk (NewContent field is byte-identical to os.ReadFile after the call). - TestSmartEditTool_MultiFile_AllOrNothing (one block fails; all files unchanged). - TestSmartEditTool_BinaryFile_Refused. - TestSmartEditTool_LargeFile_ErrFileTooLarge. - TestSmartEditTool_LargePrompt_ErrPromptTooLarge. - TestSmartEditTool_DryRun_DoesNotWrite (Phase 1 only; sha-before == sha-after). - TestSmartEditTool_LastResultPopulated. - TestSmartEditTool_DoesNotLogDiffAtInfo (CONST-042; uses zaptest.NewLogger with zapcore.InfoLevel).

Slash (edit_command_test.go): - TestEditCommand_NilInspector_ReportsUnavailable. - TestEditCommand_Status_NoAttempt_ReportsNone. - TestEditCommand_Status_LastAttempt_RendersTable. - TestEditCommand_Diff_LastAttempt_PrintsDiffs. - TestEditCommand_DryRun_FileNotFound_ReportsError. - TestEditCommand_DryRun_ValidPrompt_ReturnsDiff_NoWrite. - TestEditCommand_Commit_ValidPrompt_Writes. - TestEditCommand_Usage_OnUnknownSub.

6.2 Integration (`//go:build integration`)

`tests/integration/smartedit_test.go`:

- `TestSmartEdit_SingleFile_ContentChanged` — ALWAYS runs (no infrastructure dep). Real tempdir; writes a Go source file; runs the tool; asserts the file's sha changed AND `result.Files[0].NewContent == os.ReadFile(path)`.
- `TestSmartEdit_SearchNotFound_FileUnchanged` — sha-before == sha-after.
- `TestSmartEdit_MultiFile_AllChanged` — three files, three blocks; all sha values change.
- `TestSmartEdit_MidCommitFailure_AllFilesUnchanged` — three files; the third's parent directory is chmod'd to 0500 (read-only) before commit; asserts ALL THREE files are byte-identical to the pre-content. Cleanup restores 0700.
- `TestSmartEdit_DiffMatchesDiffU` — runs `diff -u` via `exec.CommandContext` on the captured before/after content; asserts the tool's `Diff` matches.
- `TestSmartEdit_Ambiguous_FileUnchanged` — SEARCH appears 3× in the file; tool fails; sha unchanged.
- `TestSmartEdit_Binary_Refused` — file with NUL byte in first 8 KiB; tool fails; sha unchanged.
- `TestSmartEdit_DryRun_NeverWrites` — invokes `inspector.DryRun`; asserts sha unchanged.
- `TestSmartEdit_OrderedBlocks_LaterSeesEarlier` — two blocks, same file; the second's SEARCH is the first's REPLACE; tool succeeds; final content matches the second REPLACE.

6.3 Challenge (`challenges/p1-f17-smart-file-editing/`)

Seven-phase output skeleton (every phase always runs; no gating):

```
=== SINGLE-FILE-SUCCESS (always runs) ===
[PASS] tempdir created at /tmp/p1f17-XXXX
[PASS] wrote sample.go (sha-before: <hex>)
[PASS] smart_edit tool applied 1 block, Applied=true
[PASS] re-read sample.go (sha-after: <hex>); sha-before != sha-after
[PASS] result.Files[0].NewContent matches os.ReadFile(sample.go) byte-for-
[PASS] result.Files[0].Diff is non-empty and contains the literal "@@"

=== SEARCH-NOT-FOUND-REJECTED (always runs) ===
[PASS] tempdir at /tmp/p1f17-XXXX
[PASS] wrote unchanged.go (sha-before: <hex>)
[PASS] smart_edit tool returned ErrSearchNotFound (Applied=false)
[PASS] re-read unchanged.go (sha-after: <hex>); sha-before == sha-after  (

=== MULTI-FILE-ATOMIC (always runs) ===
[PASS] tempdir at /tmp/p1f17-XXXX
[PASS] wrote a.go b.go c.go d.go e.go (sha-before captured)
[PASS] smart_edit tool applied 5 blocks across 5 files, Applied=true
[PASS] re-read all 5 files; every sha-after differs from sha-before
[PASS] result.Diff contains 5 unified-diff sections

=== PARTIAL-FAILURE-ROLLBACK (always runs) ===
[PASS] tempdir at /tmp/p1f17-XXXX
```

```
[PASS] wrote a.go b.go c.go d.go e.go (sha-before captured)
[PASS] chmod 0500 on parent dir of c.go (force write failure on c.go)
[PASS] smart_edit tool returned commit error (Applied=false)
[PASS] re-read all 5 files; every sha-after equals sha-before  (atomic rol
[PASS] cleanup restored 0700 on parent dir

=== DIFF-EXACTNESS (always runs) ===
[PASS] tempdir at /tmp/p1f17-XXXX
[PASS] smart_edit tool applied a known SEARCH/REPLACE
[PASS] independently ran `diff -u pre.txt post.txt`
[PASS] tool.Diff is byte-identical to diff -u output (after trailing-newli

=== AMBIGUOUS-REJECTED (always runs) ===
[PASS] wrote ambig.go containing SEARCH literal 3 times
[PASS] smart_edit tool returned ErrSearchAmbiguous (Applied=false)
[PASS] re-read ambig.go; sha-after == sha-before

=== BINARY-REFUSED (always runs) ===
[PASS] wrote bin.dat with NUL byte at offset 16
[PASS] smart_edit tool returned ErrBinaryFile (Applied=false)
[PASS] re-read bin.dat; sha-after == sha-before

SUMMARY: SINGLE=6/6 PASS; NOT-FOUND=4/4 PASS; MULTI=5/5 PASS; ROLLBACK=6/6
```

The Challenge MUST exit non-zero on any assertion failure. SHA mismatches (`Applied=true` but file unchanged, or `Applied=false` but file changed) are hard failures. Anti-bluff smoke clean check appended to harness output.

7. Cross-platform

Pure Go (no CGO); pure stdlib filesystem (`os.ReadFile` / `os.WriteFile`). Runs natively on `linux/amd64`, `linux/arm64`, `darwin/amd64`, `darwin/arm64`. Windows: the multiedit `BackupManager` already handles Windows path semantics; F17 inherits. The integration test that uses chmod 0500 is gated on `runtime.GOOS != "windows"` with SKIP-OK marker `SKIP-OK: P1-F17 chmod-rollback test requires unix permission semantics (Windows)`.

The cross-compile `make prod` target (linux/macos/windows) is exercised in T08.

8. Out of scope (deferred)

- **Fuzzy SEARCH matching** (typos in old text; whitespace-insensitive match) — F17.5. The lenient model is "re-search at apply time"; fuzzy is an additional layer.
- **Regex SEARCH patterns** — F17.5. Many users will want this; keeping v1 to literal-only keeps the failure modes obvious and the test surface small.
- **Partial-line / block-by-line matching** — F17.5.
- **Auto-fix-on-conflict** — F17.5. When SEARCH is not found in the current content but is found in a `git blame`-style historical version, propose the edit against the new content.
- **Binary diff** — F17.5. Binary files are refused in v1.
- **Structural / AST-aware edits** — F18 candidate.
- **Multi-step undo across separate prompts** — F17.5. Within a prompt, multiedit's Rollback is the undo. Across prompts, the user uses git.
- `/edit history` — F17.5. v1 keeps only the last attempt.

- **Marker-escape mechanism** — v2 (see §5.3).

9. Constitutional compliance

- **§11.9 / CONST-035** — Challenge has SEVEN phases, all always-run. Every phase records sha-before and sha-after with positive evidence. The five real-execution criteria in §5.2 each map to a unit + integration + Challenge assertion. Disk-state mismatch is a hard failure.
- **CONST-039** — Challenge at `challenges/p1-f17-smart-file-editing/` + evidence harness at `tests/integration/cmd/p1f17_challenge/main.go`. Every phase asserts disk content with sha-256.
- **CONST-042 (No-Secret-Leak)** — full diff text is logged at DEBUG only; INFO-level logs carry only diff byte-count. The user-facing slash output is by user request and not logged. A unit test asserts `zaptest` at `InfoLevel` does NOT see the diff body. The `Challenge` harness records sha + line count, never diff text.
- **CONST-043 (No-Force-Push)** — close-out task pushes to all four remotes non-force; explicit user authorization is requested at T10 before pushing.
- **No-Mocks-In-Production (Universal Rule 2)** — every real disk operation goes through `os.ReadFile` / `os.WriteFile` and the existing F08 multiedit transactional layer. The only test seam is `SmartEditInspector` (interface used by the slash command for testability); `SmartEditTool` is the production impl. No filesystem abstraction; no mocked disk.

10. Open questions resolved

Q	Answer	Resolution
Q1: search-replace block format	(B) standard delimiter triplet	`<<<<<<< SEARCH\n<old>\n=======\n<new>\n>>>>>>> REPLACE` per claude-code/aider convention
Q2: verification	(C) re-read + diff	After every successful Commit, re-read each file from disk and compute the unified diff between pre-apply and post-write content; both returned to the caller for self-check
Q3: atomicity	(B) build on multiedit	`SmartEditCommit` wraps the existing F08 `MultiFileEditor`; non-breaking; multi-file atomicity inherited; per-file failure during commit triggers backup-restore rollback
Q4: conflict detection	(B) lenient	At apply time, re-search the SEARCH literal in the current file content; ambiguous (n>1) and not-found (n=0) both fail with clear errors; no mtime/hash strict pre-check
Q5: user surface	(A) `/edit` slash only	`/edit status / diff / dry-run <path> / commit <path>`; no cobra subcommand

---

11. Non-obvious decisions (recorded for plan-time review)

1. **Diff is `string`, not `*Diff`** — the `EditResult.Diff` and `SmartEditResult.Diff` fields are plain strings (the unified-diff text). Reason: the

LLM consumer pattern-matches the diff with `strings.Contains` / regex; pulling in the full `*multiedit.Diff` struct would force the consumer to import `multiedit` symbols just to read `.Unified`. A string is the lowest-coupling shape; the structural diff (with stats, hunks, etc.) is still available to anyone who calls `multiedit.DiffManager.GenerateDiff` directly.

2. **No filesystem abstraction; pure `os.ReadFile` / `os.WriteFile` via multiedit** — the spec deliberately does NOT introduce a `Filesystem` interface for the smart-edit tool. The multiedit package already wraps disk access through `filesystem.FileSystemTools`; adding a second seam in smart-edit would create two divergent abstractions. CONST-042 protection lives at the logging layer, not the disk layer. Tests use real tempdirs (`t.TempDir()`).
3. **Lenient conflict detection at apply time, not transaction-open time** — the spec rejects mtime / pre-checksum strict conflict detection (Q4=B). Reason: external tools (gofmt, prettier, the user's editor on save) reformat files frequently; a strict mtime check would fail every legitimate edit when an unrelated reformat happened in the millisecond between the LLM's plan and the apply. Re-searching the SEARCH literal at apply time correctly distinguishes "file reformatted but my SEARCH still matches" (success) from "file reformatted and my SEARCH no longer matches" (failure with a clear message; the LLM re-reads + re-plans).
4. **All-or-nothing across the whole prompt, not per-file** — Q3=B says transactional via multiedit. The spec extends this: even Phase 1 (in-memory apply) aborts if any single block fails on any single file. Reason: the LLM emits a coherent multi-file refactor; partial application of half a refactor is a worse state than the original. Multi-file atomicity is inherited from multiedit's Commit; whole-prompt atomicity is enforced by the Phase-1 abort gate.
5. **Marker triplet is hardcoded literal; no escape mechanism in v1** — see §5.3 for the rationale. v1 punts on escaping; v2 may revisit. The cost of getting the escape syntax wrong (collision with claude-code/aider) is higher than the cost of "indent the marker, or use fs_write" for the rare meta-edits.
6. **No multi-attempt history; only `lastResult`** — the slash command's `/edit status` and `/edit diff` operate on a single in-memory `lastResult`. Reason: F11 session resume already persists the full agent transcript including tool invocations; smart-edit history is recoverable from the session, not from a parallel in-tool history. v1 keeps the in-tool state minimal.
7. **`/edit dry-run` reads from a file path, not from inline text** — Q5=A specified the slash; the slash takes a path because TUI input rarely accepts multi-line strings cleanly. Inline-text dry-run is available via the tool itself (the LLM can call `smart_edit` with `dry_run=true` — no, wait, v1 does not expose a dry-run param on the tool; the LLM dry-runs by parsing + applying in its own context. Slash users dry-run via the file path. Tool users commit; that's the asymmetry).
8. **Binary detect lives in smartedit, not filesystem** — duplication is acceptable here because the binary-refusal contract is a smart-edit-specific guarantee (the spec says binary files are refused; that's a smart-edit decision, not a filesystem decision). If filesystem grows its own binary-detect later, both can converge on a shared helper; today, locality wins.
9. **Re-use multiedit's diff, not a new diff impl** — the spec deliberately delegates to `multiedit.DiffManager.GenerateDiff` (already in production since F08). One LCS implementation, one set of tests, one performance characteristic. The smart-edit `diff.go` is a 5-line wrapper.
10. **`SmartEditInspector` interface in the commands package, like /sandbox + / telemetry** — F14 and F16 established the pattern: the slash command package defines a small interface (here `SmartEditInspector`) that the production tool satisfies; tests use a fake of that interface. This keeps the slash testable without dragging in the multiedit transaction machinery.